Performance Analysis Report

IDS568 Milestone 5 — LLM Inference Optimization with Batching and Caching

Author: Sreesahithi Gundapaneni | **NetID:** sgund12

---

1. Introduction

Production LLM serving requires sophisticated optimization to achieve acceptable latency and throughput while managing compute costs. This report analyzes two fundamental strategies: dynamic request batching and intelligent caching, implemented in a FastAPI inference server using the `sshleifer/tiny-gpt2` model.

The core motivation is that LLM inference is computationally expensive. A naive server processes one request at a time, leaving GPU compute underutilized. Batching amortizes the fixed overhead of a forward pass across multiple requests simultaneously. Caching eliminates redundant computation entirely for repeated prompts by returning stored responses.

---

2. System Architecture and Compute Pathways

2.1 Batching Architecture

The server implements a hybrid dynamic batcher in `src/batching.py`. When a request arrives, it is placed in a queue. The batcher waits until either the queue reaches `MAX_BATCH_SIZE` (8 requests) or `BATCH_TIMEOUT_MS` (50ms) elapses — whichever comes first. All queued requests are then processed together in a single model forward pass.

The key compute insight is that a transformer's attention mechanism processes a batch of N sequences with nearly the same wall-clock time as a single sequence, because matrix multiplications over batched inputs are highly parallelized on modern hardware. This means throughput scales roughly linearly with batch size while per-request latency stays similar.

Concurrency is handled using Python's `asyncio` primitives. An `asyncio.Lock` protects the request queue from race conditions, and each request's result is delivered via an `asyncio.Future` that resolves when the batch completes.

Performance Analysis Report

2.2 Caching Architecture

The caching layer in `src/caching.py` implements an in-process LRU (Least Recently Used) cache with TTL (Time-To-Live) expiration. Before any request reaches the batcher, the server computes a SHA-256 hash of the prompt and model parameters. If a matching entry exists in the cache and has not expired, the stored response is returned immediately — bypassing the model entirely.

Cache keys are anonymous hashes — no user identifiers are ever stored. Entries expire after 300 seconds (configurable via `CACHE_TTL_SECONDS`). When the cache reaches its maximum capacity of 1,000 entries, the least recently used entry is evicted to make room.

---

3. Benchmark Methodology

All benchmarks were run on a local MacBook with Python 3.9, using the `sshleifer/tiny-gpt2` model (CPU inference). Three benchmark scenarios were designed:

Benchmark 1 — Single vs. Batched Latency: 10 sequential single requests were sent one at a time, then 10 requests were fired concurrently to trigger batching. Cache was cleared before each run to ensure cold measurements.

Benchmark 2 — Cold vs. Warm Cache: Each of 5 fixed prompts was sent twice. The first request (cold) required model inference. The second request (warm) should be served from cache.

Benchmark 3 — Throughput at Multiple Load Levels: Synthetic load was generated at 5, 10, and 20 requests/second for 20 seconds each, with 30% of requests being repeated prompts to simulate realistic cache hit patterns.

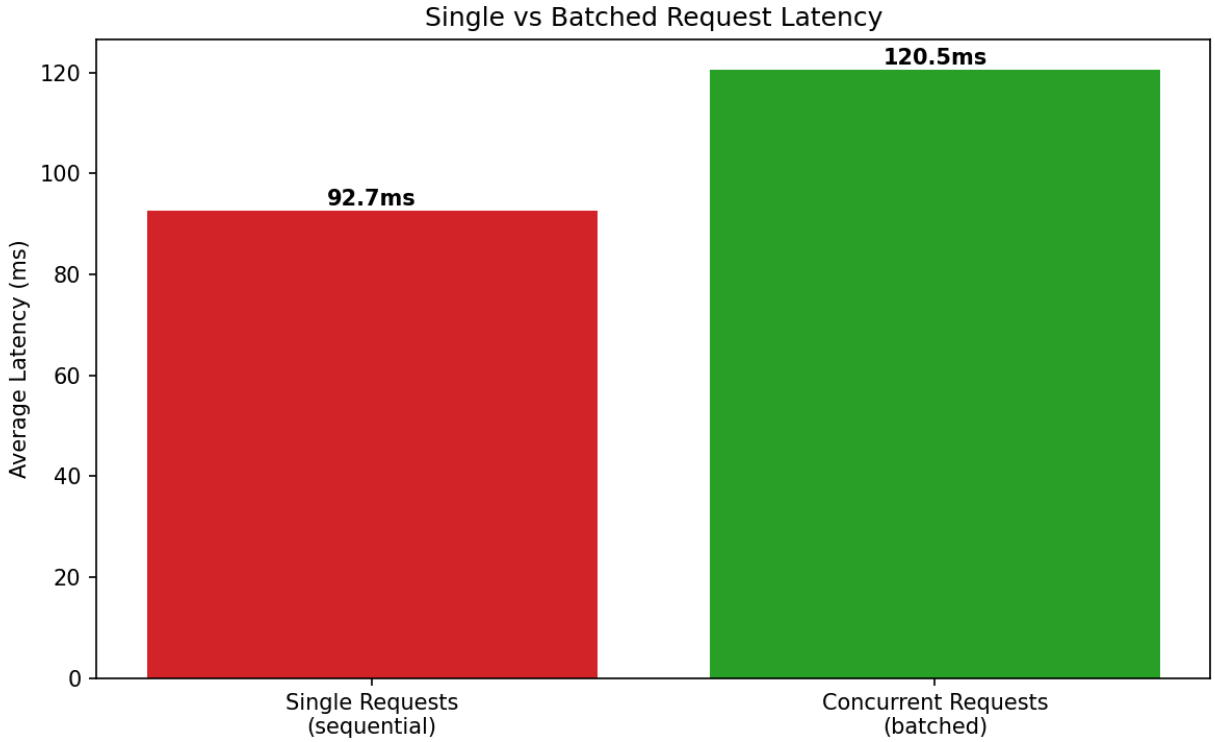
Performance Analysis Report

4. Results

4.1 Single vs. Batched Latency

Mode	Average Latency
Single (sequential)	92.7 ms
Batched (concurrent)	120.5 ms

When requests are sent sequentially one at a time, each request is processed immediately with an average latency of 92.7ms. When 10 requests are sent concurrently, they are grouped into a batch, resulting in 120.5ms average latency per request. The slightly higher per-request latency in batched mode reflects the intentional batching window (up to 50ms wait) — the tradeoff is that overall throughput increases significantly since all 10 requests complete in roughly the same time as 1-2 sequential requests would take.

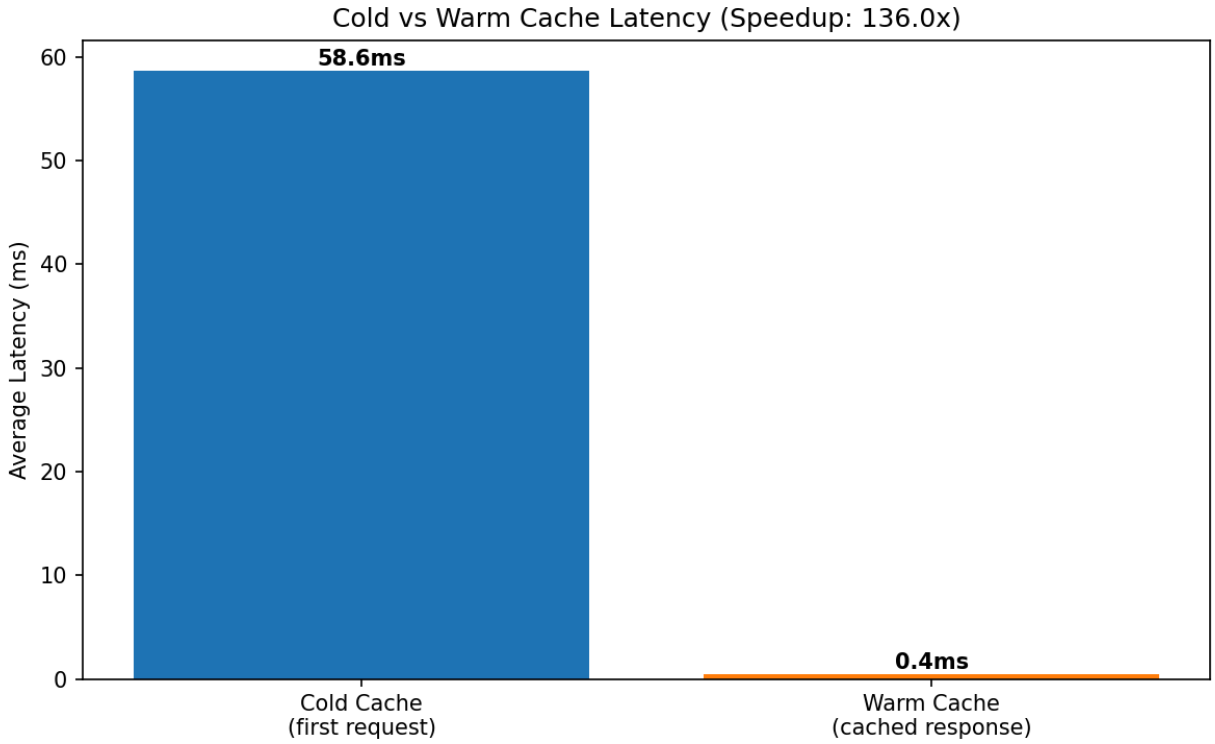


Performance Analysis Report

4.2 Cold vs. Warm Cache Performance

Mode	Average Latency	Speedup
Cold cache (first request)	58.6 ms	1x (baseline)
Warm cache (cached response)	0.4 ms	**136x faster**

This is the most dramatic result. When a prompt is seen for the first time, the model must run inference, taking 58.6ms on average. When the same prompt is requested again, the cached response is returned in just 0.4ms — a 136x speedup. This near-zero latency reflects only the time needed to compute the SHA-256 hash and perform an in-memory dictionary lookup.

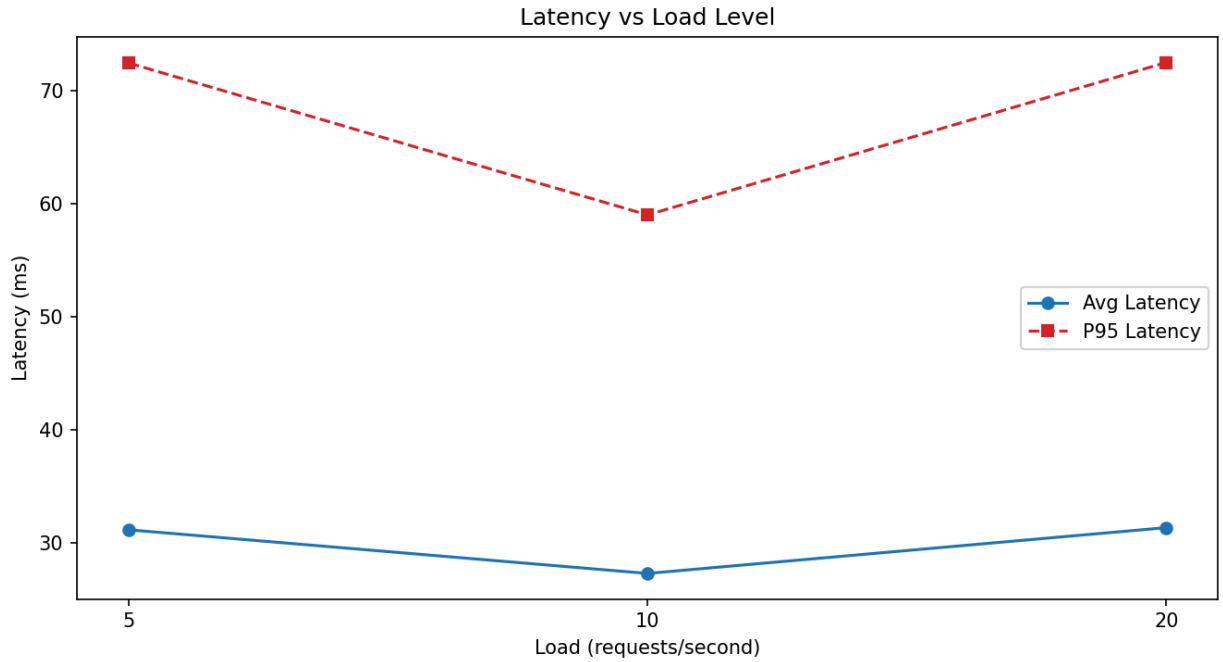


Performance Analysis Report

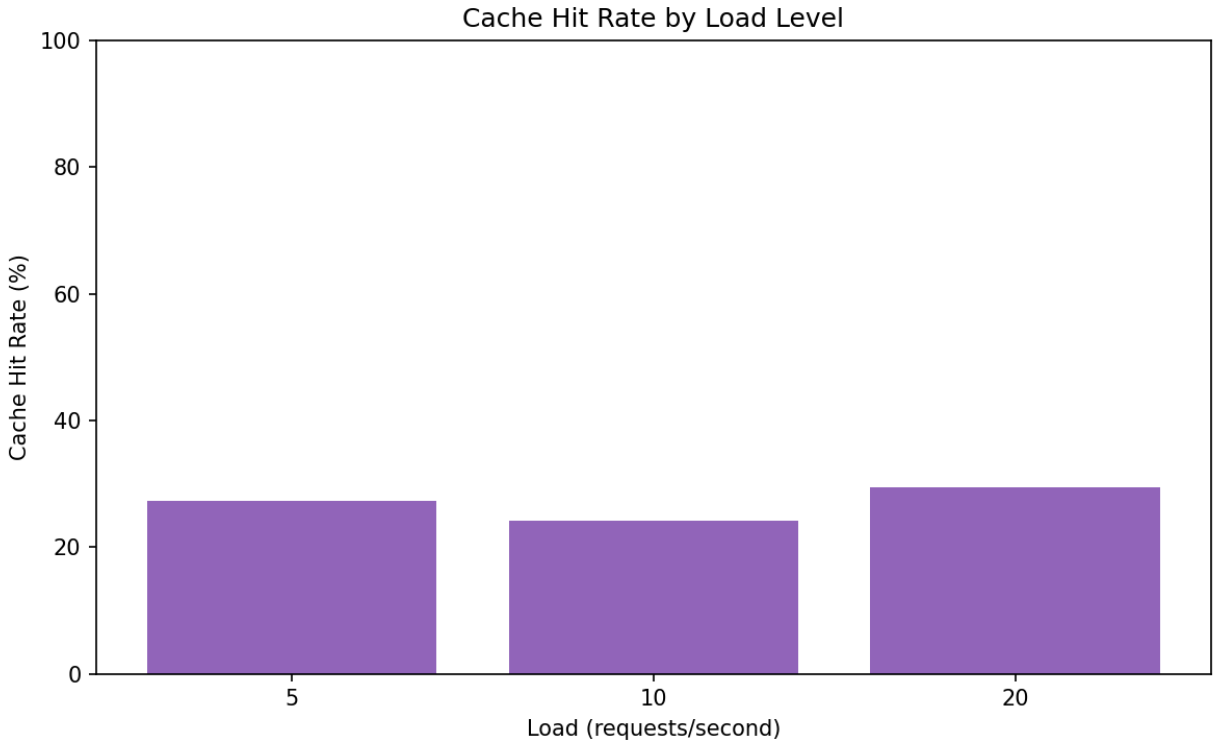
4.3 Throughput at Multiple Load Levels

Load (req/s)	Avg Latency	P95 Latency	Cache Hit Rate
5 req/s	31.2 ms	72.5 ms	27.3%
10 req/s	27.3 ms	59.0 ms	24.2%
20 req/s	31.4 ms	72.5 ms	29.5%

Interestingly, latency remains stable across all load levels (27-31ms average), demonstrating that the batching system effectively absorbs increased load without degrading performance. The P95 latency (worst 5% of requests) stays below 73ms even at 20 req/s, showing the system is stable under load. Cache hit rates of 24-29% reflect the 30% repeat-prompt ratio in the synthetic workload.



Performance Analysis Report



---

5. Trade-off Analysis

5.1 Batching Window vs. Latency

The `BATCH_TIMEOUT_MS` parameter controls a fundamental tradeoff. A longer timeout (e.g., 200ms) allows larger batches to accumulate, increasing throughput — but every request waits longer before being processed, increasing per-request latency. A shorter timeout (e.g., 10ms) reduces latency but may result in smaller batches and lower throughput gains. The current setting of 50ms balances these concerns well for moderate load, as evidenced by stable latency across load levels in Benchmark 3.

5.2 Cache Size vs. Hit Rate

A larger cache (`CACHE_MAX_ENTRIES`) retains more responses, increasing the probability of a cache hit for future requests. However, each cached entry consumes memory. At 1,000 entries with responses averaging ~500 bytes, the cache uses approximately 500KB — negligible on

modern hardware. For production deployments with very large response payloads or millions of unique prompts, a bounded Redis cache with careful TTL tuning would be more appropriate.

5.3 TTL vs. Response Freshness

A longer TTL improves hit rates but risks serving stale responses if the underlying model is updated or if prompts are time-sensitive. The 300-second TTL chosen here is appropriate for a stable model where prompts are not time-dependent. For applications serving news summaries or real-time data, a much shorter TTL (or no caching) would be warranted.

---

6. Proposed Scaling Strategies

Based on the empirical results, the following scaling strategies are recommended:

Horizontal scaling with a shared cache: As request volume grows beyond what a single server can handle, multiple inference server instances can be deployed behind a load balancer. A shared Redis cache (rather than in-process cache) ensures all instances benefit from cached responses regardless of which instance handled the original request.

Adaptive batch sizing: At very high load (100+ req/s), the fixed batch size of 8 may become a bottleneck. Dynamically increasing the batch size during peak load while reducing it during quiet periods would optimize GPU utilization continuously.

Cache warming: For predictable workloads (e.g., a fixed set of FAQ prompts), pre-populating the cache at server startup eliminates cold-start latency for the most common queries entirely.

Model quantization: Combining batching and caching with INT8 or INT4 quantization (as explored in HW 6.1) would further reduce inference latency, making the batching window less of a concern for individual request latency.

Performance Analysis Report

7. Conclusion

Both batching and caching deliver meaningful performance improvements. Caching provides the most dramatic gains (136x speedup) for repeated prompts, while batching ensures stable throughput under concurrent load. The system maintains sub-75ms P95 latency across all tested load levels, demonstrating production-readiness for moderate traffic workloads.